HydraMarker: Efficient, Flexible, and Multifold Marker Field Generation

Mingzhu Zhu, Bingwei He, Junzhi Yu, *Fellow, IEEE*, Fusong Yuan, and Jiantao Liu

Abstract—An n-order marker field is a special binary matrix whose $n \times n$ subregions are all distinct from each other in four orientations. It is commonly used to guide the composing process of position-sensing markers, which can be detected and identified in a camera image with very limited scope or severe visibility problems. Despite the advantages, position-sensing markers are rare and overlooked because generating marker fields is difficult. In this article, we broaden the definition of marker field, making it more powerful and flexible. Then, we propose bWFC (binary wave function collapse) and its high-speed version, fast-bWFC, to solve the generation problem. The methods are packaged into an open-sourced toolkit named HydraMarker, with which, users not only can generate marker fields on laptops within a short period of time, but also can highly customize them: preset values; fields and subregions in any shape; multifold local uniqueness. Comparative results indicate that the proposed method has superior efficiency, quality, and capability. It makes marker field generation accessible to common marker designers, opening up more possibilities for fiducial markers.

Index Terms—De bruijn torus, fiducial marker, HydraMarker, marker field generation, self-identifying, wave function collapse

1 Introduction

Fiducial markers are popular in computer vision applications, including but not limited to calibration [1], registration [2], robot navigation [3], surgical navigation [4], visual servo [5], AR (augmented reality) [6], and motion-tracking [7]. Compared to natural features, the artificial features they provide are easier to detect and identify, thus offering the benefit of speediness and robustness.

An evolution route of fiducial markers driven by limited camera scope and visibility problems is illustrated in Fig. 1. In controlled environments, such as single camera calibration, markers are fully observed, thus features can be identified based on their orders. In this case, repeated entities are sufficient. In less controlled situations, some features are attached with coded patterns, such as dots and sectors. When the marker is not fully presented, these coded features act as landmarks, thus nearby features can be identified. When the situation is even worse, such as in heavy occlusion, where nearby features cannot be reliably associated with coded patterns, it is necessary to code every feature. Such markers are termed composite marker fields, where the coded patterns they contain are termed ID tags.

Many technologies can provide coded patterns with good properties, such as perspective support and error tolerance [30]. Some patterns provided and detected by learning-based methods can even withstand severe deformation, off-axis viewing, and motion blur [16].

Composite marker fields can be created by stacking together individual self-identifying markers (coded patterns) as in Fig. 1c. However, this is not an efficient use of space and introduces problems if the camera field of view is small. Imagine a remote control vehicle sliding on the marker, and locating itself by a bottom view camera, whose field-of-view is slightly smaller than 2×2 ID tags. As shown in the second row and the first column of Fig. 2, the vehicle will lose location every time when it parks on a seam. In other words, naive stacking can not withstand very limited scope and severe visibility conditions. It is possible that no ID tag is fully captured even though the scope is large enough to contain one. The direct solution is by jointing the ID tags, making them seamlessly overlapped. Following the work of Burns and Mitchell [31], we name this kind of marker as position-sensing marker.

In practice, it is hard to joint ID tags together, and most coded patterns are not joinable, thus position-sensing markers are usually composed through other technical routes. Some are delicately handcrafted. For example, Matsunaga et al. [28] proposed a position-sensing chessboard whose every 4×4 block has a unique cross-ratio pair. The cross points are features, and the blocks are ID tags. Li et al. [29] provided a random noise based marker. Features and their ID tags are extracted by SURF [32]. In these two markers, ID tags are seamlessly jointed. However, both the

- *Mingzhu Zhu and Bingwei He are with the Department of Mechanical Engineering, Fuzhou University, Fuzhou 350000, China. E-mail: {mzz, mebwhe}@fzu.edu.cn.*
- *Junzhi Yu is with the State Key Laboratory for Turbulence and Complex Systems, Department of Advanced Manufacturing and Robotics, College of Engineering, Peking University, Beijing 100871, China. E-mail: junzhi.yu@ia.ac.cn.*
- *Fusong Yuan is with the National Engineering Laboratory for Digital and Material Technology of Stomatology, Center of Digital Dentistry, Peking University School and Hospital of Stomatology, Beijing 100190, China. E-mail: yuanfusong@bjmu.edu.cn.*
- *Jiantao Liu is with the Department of Mechanical Engineering, Fuzhou University, Fuzhou 350000, China, and also with the Xiamen Port Holding Group co. Ltd, Xiamen, Fujian 361000, China. E-mail: liujiantao@gmail.com.*

Manuscript received 30 March 2022; revised 27 August 2022; accepted 4 October 2022. Date of publication 10 October 2022; date of current version 3 April 2023.
This work was supported in part by the National Key Research and Development Program of China under Grant 2020YFB1312800, and in part by the National Natural Science Foundation of China under Grant 62003007.
(Corresponding author: Junzhi Yu.)
Recommended for acceptance by W. Heidrich.
Digital Object Identifier no. 10.1109/TPAMI.2022.3212862

0162-8828 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Lakehead University. Downloaded on September 28,2026 at 21:34:37 UTC from IEEE Xplore. Restrictions apply.

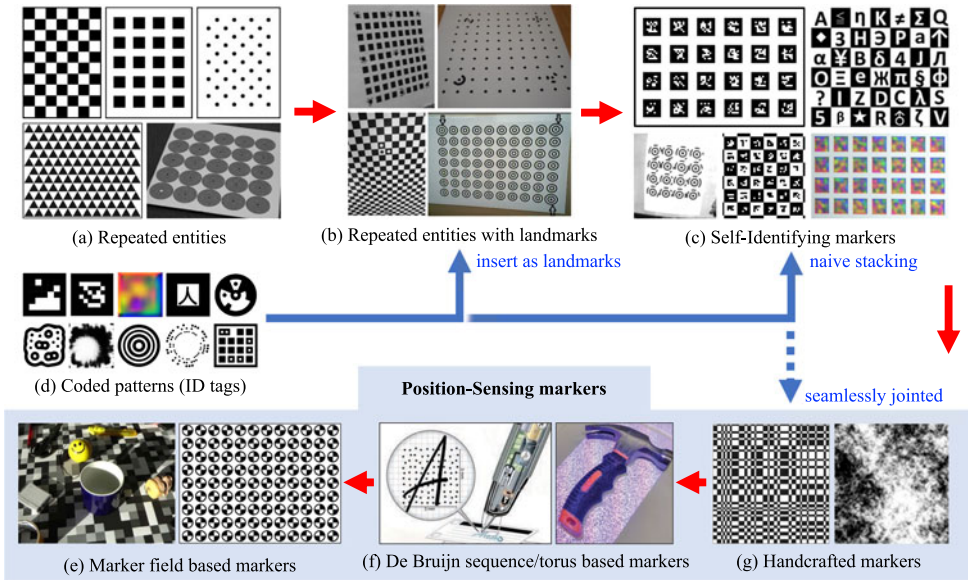


Fig. 1. An evolution route of fiducial markers driven by limited camera scope and visibility problems. From top-left to bottom-right, (a) Repeated entities, they are cross points, squares, dots, deltilles [8], and concentric circles [9]; (b) Repeated entities with landmarks, they are coded squares, circles, and dots [10], and the pattern from DLR CalLab [11]; (c) Self-Identifying markers, they are CALTag [12], symbols pattern [13], fully coded circles [14], stacked ARTag [15], and stacked DeepFormableTag [16]; (d) Coded patterns, they are Matrix [17], ARTag [18], DeepFormableTag [16], ArUco [19], InterSense [20], reacTIVision [21], E2ETag [22], CCTag [23], RuneTag [24], and TopoTag [25]; (e) Marker field based position-sensing markers, they are Herout et al. [26] and Gao et al. [27]; (f) De Bruijn sequence/torus based position-sensing markers used in digital papers and structured light; (g) Handcrafted position-sensing markers, they are Matsunaga et al. [28] and Li et al. [29].

invariance of cross-ratio and the subpixel accuracy of SURF are sensitive to distortion.

An easier way of composing position-sensing markers is referring to de Bruijn sequence/torus [33], [34]. A de Bruijn sequence is a 1D string whose every length-n substring is unique, and a de Bruijn torus is a 2D matrix whose every $m \times n$ sub-matrix is unique. They have derived some commercially successful products. For example, the dot arrays of digital papers are position-sensing markers. Each dot is printed near a grid intersection with a slight offset in one of the four directions, thus the whole dot array represents a 4-state matrix. This matrix is commonly built based on the cross of two 6-order 2-state de Bruijn sequences. The digital pen estimates its absolute position on this paper based on the 6×6 local regions observed by a bottom view camera. Another example can be found in several structured light based scanners, where the projected patterns are position-sensing markers mapped from de Bruijn torus.

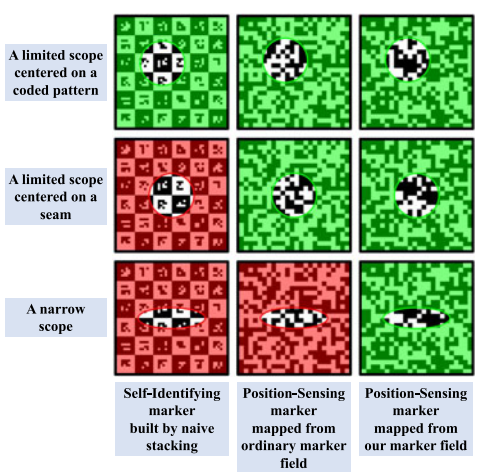


Fig. 2. Position-Sensing ability. From left to right, a self-identifying marker; a position-sensing marker based on a 4-order marker field, whose every 4×4 block is unique; a position-sensing marker based on the proposed marker field, whose every 4×4, 3×6, 2×9, or 1×20 subregion is unique. The masks indicate the camera scopes, and their colors indicate whether an ID tag is fully contained (green) or not (red).

An essential problem of de Bruijn sequence/torus is that, the sub-string/sub-matrix is only unique in one direction, thus corresponding position-sensing markers lack orientability [31]. To solve this problem, Szentandrasi et al. [35] proposed marker field, which can be regarded as aperiodic 4-orientable de Bruijn torus. Specifically, an n-order marker field is a binary matrix whose $n \times n$ subregions are unique in all four orientations. Therefore, corresponding position-sensing markers embed both position and orientation information. For example, the edge directions of Herout et al. [26] and the cross point orientations of Gao et al. [27] are coded based on 4-order marker fields. Both have achieved satisfactory performance in their applications, where Herout et al. [26] suffers from heavy occlusions and Gao et al. [27] suffers from very limited scopes.

Now, the challenge of composing position-sensing markers becomes *"how to generate a marker field?"*. Narita et al. [36] used brute-force searching. Gao et al. [27] made an index table of all possible ID tags, and grew a marker field by linking them iteratively, which is basically an accelerated brute-force method. Both of them are limited in small size, thus can not satisfy the applications that require position-sensing markers with large amounts of coded features, such as the two applications of de Bruijn sequence/torus in Fig. 1. Currently, the most promising generation method is Szentandrasi et al. [35], which has achieved a 4-order marker field

Authorized licensed use limited to: Lakehead University. Downloaded on September 28,2026 at 21:34:37 UTC from IEEE Xplore. Restrictions apply.

with 110×78 size[1]. However, it is still inconvenient because of the high computing cost and the lack of flexibility.

In this paper, we propose a novel marker field generation method and its fast version. They are packaged into a toolkit named HydraMarker[2]. It has three advantages:

1) *Efficient*. It is significantly faster than existing methods. Marker designers can build their marker fields on laptops within minutes, rather than waiting for weeks or finding a supercomputer.
2) *Flexible*. Compared to existing methods whose marker fields must be rectangular and ID tags must be square, it allows arbitrary shapes. Furthermore, the values of marker field can be preset. Marker designers can highly customize the marker field for their applications.
3) *Multifold*. Existing methods only allow one tag shape for one marker field, while HydraMarker allows multiple tag shapes simultaneously. This implies that the fields can have multifold local uniqueness, thus the derived position-sensing markers can work with extreme visibility problems (see the last row of Fig. 2).

The marker field generated by HydraMarker is no longer an improved version of de Bruijn torus. It is beyond the original definition. However, we still use this name because this work still serves the original purpose [35]. When necessary, we use advanced marker field to refer to flexible and multifold marker field, and ordinary marker field to refer to rectangle marker field with square ID tags.

The paper is arranged in the following ways. Section 2 introduces HydraMarker toolkit. Section 3 proposes the generating algorithm and Section 4 offers its fast version. Section 5 makes comparisons. Section 6 gives the conclusion.

2 HydraMarker

HydraMarker is a toolkit to generate advanced marker fields, which are binary matrices with a key property that *"every subregion in any tag shape is unique in all four orientations"*. Besides 0 and 1, the matrices have a special value h, which means invalid or not interested (as in *h*ollow).

Similar to de Bruijn sequence/torus, marker fields are nothing more than organized numbers. They are independent of marker appearance. Users are free to design marker appearance for their applications. Examples are displayed in Fig. 3. If the state grid is available for reading, Fig. 3b is an efficient appearance of the marker field in Fig. 3a; If the marker is attached to a curved surface, Fig. 3c is a considerable option; If a high accuracy of feature location is required, we can use Fig. 3d alternatively. It is also feasible to concatenate two marker fields into a color position-sensing marker, as shown in Fig. 3e. We hide a red smiling face in this color marker, and the face is part of the position-sensing code. It does not affect the 4-orientable local uniqueness. Neither de Bruijn sequence/torus nor ordinary marker field has such customizability. HydraMarker can also be used in a way similar to de Bruijn sequence. In Fig. 3f, a color marker with 4 states is mapped from the cross of two 1D marker fields.

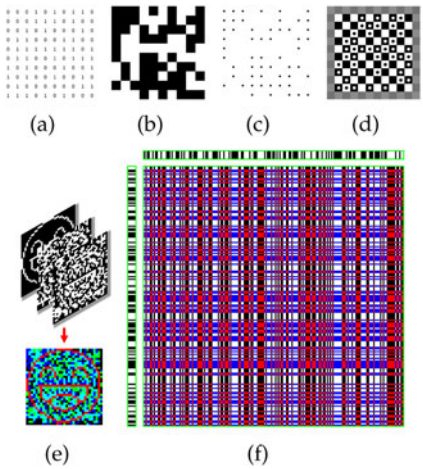


Fig. 3. Compose position-sensing markers based on marker fields.

The inputs of HydraMarker toolkit consist of an initial marker field and several tag shapes. The initial marker field must have the same size with the desired output marker field. Users can shape the initial marker field by assigning h to not interested elements, and can also preset several elements with 0 or 1. The remaining elements are u (as in $u$nknown). Tag shapes are defined by binary matrices too, but here 0 means empty and 1 means solid.

During marker generation, HydraMarker gradually replaces the u in the initial marker field with 0 or 1. Note that, the desired marker field might not exist. If HydraMarker warns that the required size is too large or rolls back frequently but makes no progress, users should consider smaller marker fields, fewer presets, and larger tag shapes.

Examples of using HydraMarker are illustrated in Fig. 4. In the first example, four tag shapes are used simultaneously to ensure the survival of at least one ID tag under non-ideal visibility conditions. The initial marker field is a 10×10 matrix full of u. The output marker field and a corresponding fiducial marker are displayed on the right. In the second example, we want to mark a base plane for an AR application. Since objects usually occupy the middle area of the base, a frame-shape marker field is requested. The initial marker field is a 10×10 matrix with h in the middle and u near the boundaries.

The reading process is up to users because it closely relates to the designed marker appearances. We provide an example based on the appearance of Fig. 3d. The code is open-sourced along with the toolkit, but it will not be introduced here because it is out of the scope of this paper. Several examples are displayed in Fig. 5. As can be seen, visibility problems might be introduced by splitting, occlusion, noise, or disturbance. However, whatever the reason, one survived ID tag is sufficient to enable the application.

3 Generating Algorithm

Generating a marker field is not easy because the following constraints must be satisfied:

1) *No self-conflict*. ID tags cannot be the same with themselves after 90°/180°/270°-rotation.

1. http://www.fit.vutbr.cz/research/groups/graph/MF/
2. https://github.com/Lilin2015/Author—HydraMarker

Authorized licensed use limited to: Lakehead University. Downloaded on September 28,2026 at 21:34:37 UTC from IEEE Xplore. Restrictions apply.

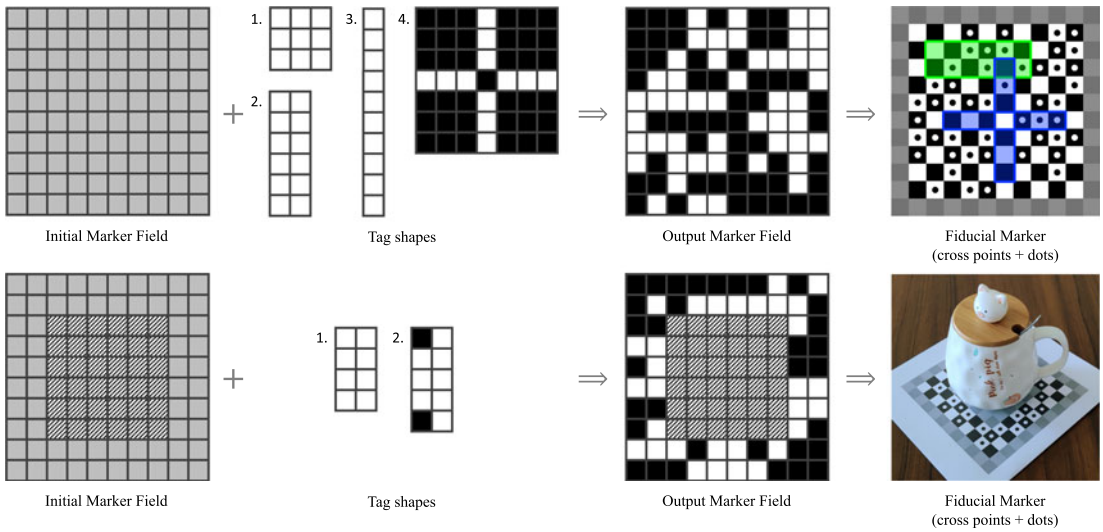


Fig. 4. Using HydraMarker. Black, white, gray, or slashed squares mean the element equals 0, 1, u, or h, respectively.

2) *No cross-conflict*. ID tags cannot be the same with others after $0°/90°/180°/270°$-rotation.
3) *No seam*. ID tags exist in every subregion with any pre-defined tag shape from any orientation.

The last constraint is the most challenging one. It means that every element in the marker field affects multiple ID tags simultaneously. Assigning the elements is like solving a complex Sudoku.

In this article, we propose bWFC (binary wave function collapse) to generate marker field. Imagine a hidden marker field to be revealed, and its elements are in superposition states, that is, the combination of 0 and 1. Every time we reveal an element, it collapses to a certain state. The collapse also brings information of remaining hidden elements, because some possibilities can be excluded based on the three constraints. In other words, it makes the superposition states of some hidden elements unbalanced. We can predict the state of an element before we reveal it, and the chance of a wrong prediction can be minimized if we select the most unbalanced one.

Generating a marker field is filling a matrix of many unknowns (initial field with many u). It is similar to revealing a hidden marker field, as assigning every element is based on a prediction. The difference is that we do not know whether the predictions are correct until the marker field is complete, or the constraints are violated and the process is stuck. Therefore, we need to maximize the probability of every prediction being correct. This leads to the core idea of bWFC: *"it always collapses the most unbalanced unknown, thus the probability of a wrong move is minimized"*. In the following context, we first introduce several important objects and actions, and then propose the overall workflow.

3.1 Marker Field $\mathbf{F}$

bWFC starts from an initial marker field $\mathbf{F}_{\text{initial}}$. It gradually fills up unknown elements. The filling process is the cycle of collapse and propagation. In collapse, one unknown of $\mathbf{F}$ is selected and assigned a binary value. There are multiple options, and most of them lead to failure. The chance of a wrong move is minimized by collapsing the most unbalanced one to the state it leans to. This process is informed by superstate sensing metrics, including likelihood matrix $\mathbf{L}$ and support matrix $\mathbf{C}$, which are calculated based on the current marker field and a tag pool $\mathbf{P}$ storing all the legal ID tags that remain. In propagation, the effect of the latest collapse is handled by updating $\mathbf{L}$, $\mathbf{C}$, and $\mathbf{P}$, which will subsequently guide the next collapse.

In particular, bWFC can minimize the chance of a wrong move, but can not eliminate it. In the late stage of the filling process, it is not surprising that the constraints are violated no matter what value is assigned to an unknown element. In this situation, the process rolls back and tries other paths. Some elements of $\mathbf{F}$ are reset to u and reassigned later. A multiway tree T is employed to manage this process.

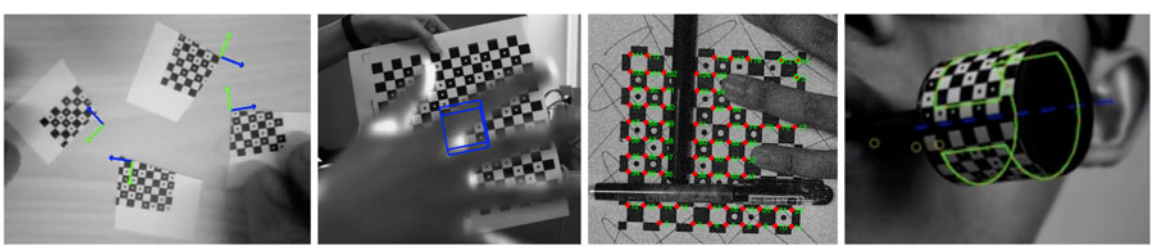


Fig. 5. Using position-sensing markers. From left to right, we use a position-sensing marker to locate the central frames of a split marker; project a virtual cube on an occluded marker; identify cross point features under severe noise, disturbance, and occlusion; calculate the pose of a localizer for dental surgery.

Authorized licensed use limited to: Lakehead University. Downloaded on September 28,2026 at 21:34:37 UTC from IEEE Xplore. Restrictions apply.

3.2 ID Tag K

An ID tag, denoted as $\mathbf{K}$, is an 8-connected subregion of $\mathbf{F}$ in predefined shapes. The shape is described in the form of binary matrices $\mathbf{\Omega}$. For example, the 2nd and the 4th tag shapes of the first example in Fig. 4 are

$$\mathbf{\Omega}_2 = \begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{bmatrix} \mathbf{\Omega}_4 = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}, \tag{1}$$

and the green and blue ID tags are

$$\mathbf{K}_{2,90}^{1358} = \begin{bmatrix} 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \end{bmatrix}$$
$$\mathbf{K}_{4,0}^{135} = \begin{bmatrix} h & h & h & 0 & h & h & h \\ h & h & h & 1 & h & h & h \\ h & h & h & 0 & h & h & h \\ 0 & 0 & 0 & h & 1 & 1 & 1 \\ h & h & h & 0 & h & h & h \\ h & h & h & 0 & h & h & h \\ h & h & h & 0 & h & h & h \end{bmatrix} \tag{2}$$

respectively, where $\mathbf{\Omega}_s$ means the sth user-defined shape. $\mathbf{K}_{s,0}^k$ is an ID tag in shape $\mathbf{\Omega}_s$, and its corresponding key value is k. The key value is the decimal form of the binary vector transformed from the ID tag (the matrix on the right side of Eq. (2)) in row-major order with h ignored. $\mathbf{K}_{s,r}^k$ is the ID tag rotated from $\mathbf{K}_{s,0}^k$ by r degree anti-clockwise. Note that, an ID tag can be explained in different orientations, thus $\mathbf{K}_{2,90}^{1358} = \mathbf{K}_{2,270}^{1834}$ and $\mathbf{K}_{4,0}^{135} = \mathbf{K}_{4,90}^{58} = \mathbf{K}_{4,180}^{3600} = \mathbf{K}_{4,270}^{1472}$.

3.3 Hit Maps H and $\tilde{\mathbf{H}}$

Full hit map $\mathbf{H}$ and partial hit map $\tilde{\mathbf{H}}$ are frequently used in bWFC, indicating what ID tags already appear and can appear in $\mathbf{F}$ respectively. $\mathbf{H}_{s,r}^k(\mathbf{x}) = 1$ means $\mathbf{F}$ contains a complete $\mathbf{K}_{s,r}^k$ centered at $\mathbf{x}$ (for convenience, the center of tag $\mathbf{K}_{s,r}^k$ is defined on the top-left element, other definitions are feasible and will not affect the final result). $\tilde{\mathbf{H}}_{s,r}^k(\mathbf{x}) = 1$ means $\mathbf{F}$ might contain a $\mathbf{K}_{s,r}^k$ centered at $\mathbf{x}$, with some elements remained unknown but all known elements match the tag. An example is displayed in Fig. 6.

Before the operations, we define binary matrices

$$\mathbf{F}^V(\mathbf{x}) = \begin{cases} 1, & \text{if } \mathbf{F}(\mathbf{x}) \in V \\ 0, & \text{others} \end{cases}$$
$$\mathbf{K}_{s,r}^{k,V}(\mathbf{x}) = \begin{cases} 1, & \text{if } \mathbf{K}_{s,r}^k(\mathbf{x}) \in V \\ 0, & \text{others} \end{cases}, \tag{3}$$

where V is a set containing one or more states of $\{0, 1, h, u\}$. For example, $\mathbf{F}^{0,u}$ is a matrix whose elements equal 1 when corresponding elements in $\mathbf{F}$ equal 0 or u.

Then, we investigate where the 0 and 1 of $\mathbf{K}$ match the 0 and 1 of $\mathbf{F}$ respectively, as

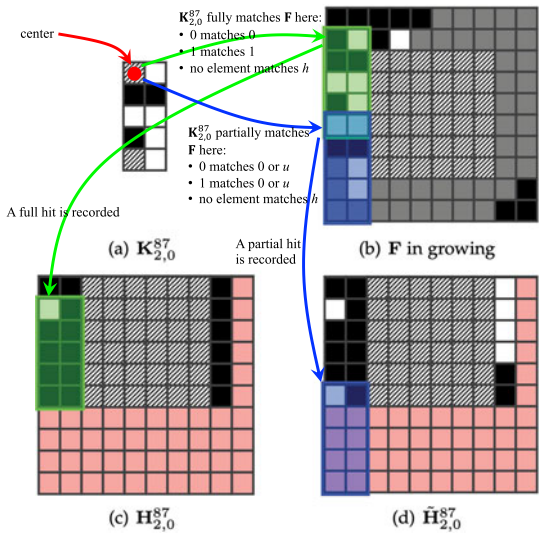


Fig. 6. An example of the hit maps in generating the second example of Fig. 4. Black, white, gray, or slashed squares mean the elements equals 0, 1, u, or h, respectively. Hit maps are smaller than $\mathbf{F}$, and the differences are marked in magenta.

$$\begin{cases} \mathbf{H}_{s,r}^{k,0} &= \mathbf{F}^0 \ominus \mathbf{K}_{s,r}^{k,0} \\ \mathbf{H}_{s,r}^{k,1} &= \mathbf{F}^1 \ominus \mathbf{K}_{s,r}^{k,1} \end{cases}, \tag{4}$$

and where $\mathbf{K}$ touches hollows, as

$$\mathbf{H}_{s,r}^{k,h} = \mathbf{F}^h \oplus (1 - \tilde{\mathbf{K}}_{s,r}^{k,h}). \tag{5}$$

In these equations, $\ominus/\oplus$ means erosion/dilation, and $\tilde{\mathbf{K}}_{s,r}^{k,h}$ is 180°-rotated $\mathbf{K}_{s,r}^{k,h}$. At last, we investigate where $\mathbf{K}$ matches $\mathbf{F}$ to obtain the fully hit map $\mathbf{H}$, as

$$\mathbf{H}_{s,r}^k(\mathbf{x}) = \begin{cases} h, & \text{if } \mathbf{H}_{s,r}^{k,h}(\mathbf{x}) = 1 \\ 1, & \text{if } \mathbf{H}_{s,r}^{k,0}(\mathbf{x}) \& \mathbf{H}_{s,r}^{k,1}(\mathbf{x}) = 1 \text{ and } \mathbf{H}_{s,r}^{k,h}(\mathbf{x}) = 0 \\ 0, & \text{others} \end{cases}. \tag{6}$$

The only difference between $\mathbf{H}$ and $\tilde{\mathbf{H}}$ is the way of treating u, as

$$\begin{cases} \tilde{\mathbf{H}}_{s,r}^{k,0} &= \mathbf{F}^{\{0,u\}} \ominus \mathbf{K}_{s,r}^{k,0} \\ \tilde{\mathbf{H}}_{s,r}^{k,1} &= \mathbf{F}^{\{1,u\}} \ominus \mathbf{K}_{s,r}^{k,1} \end{cases}, \tag{7}$$

and

$$\tilde{\mathbf{H}}_{s,r}^k(\mathbf{x}) = \begin{cases} h, & \text{if } \mathbf{H}_{s,r}^{k,h}(\mathbf{x}) = 1 \\ 1, & \text{if } \tilde{\mathbf{H}}_{s,r}^{k,0}(\mathbf{x}) \& \tilde{\mathbf{H}}_{s,r}^{k,1}(\mathbf{x}) = 1 \text{ and } \mathbf{H}_{s,r}^{k,h}(\mathbf{x}) = 0 \\ 0, & \text{others} \end{cases}. \tag{8}$$

3.4 Tag Pool P

The tag pool $\mathbf{P}$ is a table of ID tags. It is initialized by organizing all possible tags based on their shapes, orientations, and key values. A glimpse of the initial tag pool used to generate the second example of Fig. 4 is displayed in Fig. 7. It can be regarded as S arrays, and each array contains $2^{|\mathbf{\Omega}_s|} \times 4$ tags, where S is the number of tag shapes. ID tags of the

Authorized licensed use limited to: Lakehead University. Downloaded on September 28,2026 at 21:34:37 UTC from IEEE Xplore. Restrictions apply.

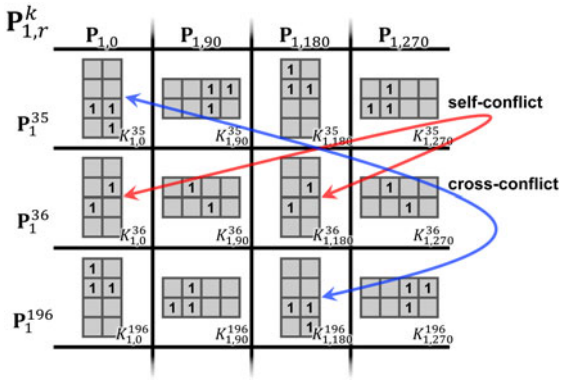


Fig. 7. An example of tag pool. Tag pool is a table of ID tags, indexed by shape, orientation, and key value.

same row have the same key value, and each is the 90°-rotated version of its former one.

Some ID tags repeat themselves after a specific rotation, denoted as self-conflicts, and some ID tags are the same as others from different rows, denoted as cross-conflicts, see Fig. 7. These tags will confuse the reading algorithm thus should never appear in $\mathbf{F}$. Note that, the rows with cross-conflicts actually have the same ID tags but in different orders. In Algorithm 1, *"remove self-conflicts and cross-conflicts from* $\mathbf{P}$*"* means removing all self-conflict rows and the redundancies of cross-conflict rows.

In the filling process, the updating tag pool $\mathbf{P}$ always stores the ID tags that can be filled into somewhere in the growing $\mathbf{F}$ without violating the first two constraints. We assume the prior of marker orientation is unavailable to reading algorithms, so they can not identify the ID tags from the same row. Therefore, for every ID tag $\mathbf{K}^k_{s,r}$ that has been included in $\mathbf{F}$, the corresponding row $\mathbf{P}^k_s$ will be banned. This process is called *"calculate* $\mathbf{P}$*"* in Algorithm 1. We can get all the tags included in $\mathbf{F}$ by querying the full hit map, as

$$\mathbf{H}^k_s(\mathbf{x}) = \sum_r \mathbf{H}^k_{s,r}(\mathbf{x}), \quad r = 0, 90, 180, 270, \tag{9}$$

where h is treated as 0 in accumulation. If $\sum_{\mathbf{x}} \mathbf{H}^k_s(\mathbf{x}) = 1$, $\mathbf{P}^k_s$ is recognized as included in $\mathbf{F}$ and banned. Sometimes a single collapse composes multiple ID tags from the same row $\mathbf{P}^k_s$. This situation can be detected based on the condition of $\sum_{\mathbf{x}} \mathbf{H}^k_s(\mathbf{x}) > 1$. The filling process will roll back when this happens.

3.5 Superstate Stack $\mathbf{N}^v$

$\mathbf{P}$ informs us what ID tags are still usable, providing their shape index s and key value k. Then, $\tilde{\mathbf{H}}^k_{s,r}$ informs us the legal locations. By stacking usable ID tags to corresponding legal locations in $\mathbf{F}$, we can have the occurrence numbers of 0 and 1 on each element, as

$$\begin{cases} \mathbf{N}^0_{s,r,\mathbf{y}}(\mathbf{x}) & = \sum_k \tilde{\mathbf{H}}^k_{s,r}(\mathbf{y}) \left(1 - \mathbf{K}^k_{s,r}(\mathbf{y} - \mathbf{x})\right) \\ \mathbf{N}^1_{s,r,\mathbf{y}}(\mathbf{x}) & = \sum_k \tilde{\mathbf{H}}^k_{s,r}(\mathbf{y}) \mathbf{K}^k_{s,r}(\mathbf{y} - \mathbf{x}) \end{cases}, \tag{10}$$

where $\mathbf{y} \in \tilde{\mathbf{\Omega}}_{s,r}(\mathbf{x})$, $\tilde{\mathbf{\Omega}}_{s,r}$ is the 180°-rotated $\mathbf{\Omega}_{s,r}$, both h and $1 - h$ are treated as 0 in multiplication. Intuitively, $\mathbf{N}^v_{s,r,\mathbf{y}}(\mathbf{x})$ indicates how many ID tags in $\mathbf{P}_{s,r}$ centered at $\mathbf{y}$ support $\mathbf{F}(\mathbf{x})$ to be v (0 or 1).

If $\mathbf{\Omega}_s$ is 90°-repeating, all $\mathbf{\Omega}_{s,r}$ are the same, such as the first tag shape in the first example of Fig. 4. In this situation, $\mathbf{N}^v_{s,0,\mathbf{y}}$, $\mathbf{N}^v_{s,90,\mathbf{y}}$, $\mathbf{N}^v_{s,180,\mathbf{y}}$, and $\mathbf{N}^v_{s,270,\mathbf{y}}$ actually refer to the same stack. A similar situation exists for 180°-repeating shape. Therefore, we further define

$$\mathbf{N}^v_{s,R_n,\mathbf{y}}(\mathbf{x}) = \sum_{r \in R_n} \mathbf{N}^v_{s,r,\mathbf{y}}(\mathbf{x}), \tag{11}$$

where

$$\begin{cases} R_1 = \{0, 90, 180, 270\}, \\ \qquad \text{if the } s^{\text{th}} \text{ shape is } 90° - \text{repeating} \\ R_1 = \{0, 180\}, R_2 = \{90, 270\}, \\ \qquad \text{if the } s^{\text{th}} \text{ shape is } 180° - \text{repeating} \\ R_1 = \{0\}, R_2 = \{90\}, R_3 = \{180\}, R_4 = \{270\}, \\ \qquad \text{if the } s^{\text{th}} \text{ shape does not repeat itself} \end{cases}. \tag{12}$$

3.6 Balance Metrics $\mathbf{L}$ and $\mathbf{C}$

The core idea of bWFC is to always collapse the most unbalanced state in $\mathbf{F}$, but how to describe balance? In this paper, we propose two metrics, likelihood $\mathbf{L}$ and support $\mathbf{C}$. Both have the same size with $\mathbf{F}$, and are defined as follows:

$$\begin{aligned} \mathbf{L}(\mathbf{x}) &= \prod_{s,R_n,\mathbf{y}} \frac{\mathbf{N}^1_{s,R_n,\mathbf{y}}(\mathbf{x})}{\mathbf{N}^0_{s,R_n,\mathbf{y}}(\mathbf{x}) + \mathbf{N}^1_{s,R_n,\mathbf{y}}(\mathbf{x})} \\ \mathbf{C}(\mathbf{x}) &= \min_{s,R_n,\mathbf{y}} \left(\mathbf{N}^0_{s,R_n,\mathbf{y}}(\mathbf{x}), \mathbf{N}^1_{s,R_n,\mathbf{y}}(\mathbf{x})\right) \end{aligned} \tag{13}$$

Intuitively, $\mathbf{L}(\mathbf{x})$ indicates whether the 0 and 1 states are balanced at $\mathbf{F}(\mathbf{x})$. $\mathbf{F}(\mathbf{x})$ has a large possibility to be 0 and 1 if $\mathbf{L}(\mathbf{x})$ is close to 0 and 1 respectively. Related stacks are considered independently, thus multiplication is used. $\mathbf{C}(\mathbf{x})$ indicates how many ID tags support the assignment of $\mathbf{F}(\mathbf{x})$. If $\mathbf{F}(\mathbf{x})$ wants to collapse to a certain state, there should be at least one support in the view of every related stack. $\mathbf{F}(\mathbf{x})$ should be collapsed as soon as possible if $\mathbf{C}(\mathbf{x})$ is low, because the collapse of other elements might take away its last support.

3.7 Tree T and the Overall Algorithm

In each collapse, bWFC finds out the unknowns with the least support based on $\mathbf{C}$, and checks their preferred states based on $\mathbf{L}$. This composes collapse options in the form of index-value pairs, as

$$\begin{aligned} Pairs' &= \{(x, \text{round}(\mathbf{L}(x)) | \mathbf{C}(x) = \min(\mathbf{C})\} \\ &= \{(x_1, v_1), (x_2, v_2), \ldots\}, \end{aligned} \tag{14}$$

where $\min(\mathbf{C})$ means the minimal value in $\mathbf{C}$. These index-value pairs are equally important, but for brief, bWFC will execute collapse based on the first element, as $\mathbf{F}(x_1) = v_1$. Furthermore, bWFC must be able to rectify wrong assignments, thus we actually use

$$Pairs = \{(x_1, v_1), (x_1, 1 - v_1)\}, \tag{15}$$

Authorized licensed use limited to: Lakehead University. Downloaded on September 28,2026 at 21:34:37 UTC from IEEE Xplore. Restrictions apply.

which is a state-first rectification, or

$$Pairs = \{(x_1, v_1), (x_2, v_2), \ldots, (x_1, 1 - v_1), (x_2, 1 - v_2), \ldots\}, \tag{16}$$

which is a position-first rectification. In practice, no meaningful difference has been found between the two. We use state-first rectification in our implementation. The first index-value pair will be removed from $Pairs$ before rectification to avoid bWFC repeating a same path.

Algorithm 1. bWFC

Input: initial marker field $\mathbf{F}_{\text{initial}}$, tag shapes $\boldsymbol{\Omega}$
Output: marker field $\mathbf{F}$

```
F = F_initial
define an empty multi-way tree T, set d = 0
initialize tag pools P based on Ω
remove self-conflicts and cross-conflicts from P
while F contains unknowns do
    calculate H, P, L, and C based on F
    if H or L is illegal then
        repeat
            d ← d − 1
            F(T_d^p.index) ← u
            remove T_d^p from T_d
        until T_d is nonempty
    else
        calculate Pairs, and assign to T_d
    end if
    T_d^p ← the first node of T_d
    F(T_d^p.index) ← T_d^p.value
    d ← d + 1
end while
```

There are two conditions triggering rolling back. The first one is when full hit map reports conflicts, as $\sum_{\mathbf{x}} \mathbf{H}_s^k(\mathbf{x}) > 1$ (see Section 3.4), and the second one is when a denominator of $\mathbf{L}(\mathbf{x})$ equals zero (see Eq. (13), it means neither 0 nor 1 is available to $\mathbf{F}(\mathbf{x})$). In Algorithm 1, these conditions are checked at *"if* $\mathbf{H}$ *or* $\mathbf{L}$ *is illegal"*.

A searching strategy is required to distinguish rectification from a first-time assignment, and $Pairs$ is calculated only at first time assignment. This is realized based on a multi-way tree T, whose nodes are the index-value pairs. The algorithm is illustrated in Algorithm 1, where d is an iterator always equaling the number of settled unknowns. T_d is the nodes at depth d of T, and T_d^p is the parent node of T_{d+1}.

An example of generating 5×5 marker field with 1×5 stripe tag shape is illustrated in Fig. 8. Four snapshots of $\mathbf{F}$, $\mathbf{L}$, and $\mathbf{C}$ are displayed, they are at $d = 1$, 9, 11, and 25, respectively. When $d = 1$, the top-left element has collapsed to 0, $\mathbf{C}$ suggests to collapse the two opposite elements in the next step, and $\mathbf{L}$ suggests to assign them with 1. This is reasonable because the risk of self-conflict increases if the top-right and bottom-left elements are 0 too. When $d = 9$, $\mathbf{C}$ indicates that the element at (4,2) is safer, thus can be considered later. When $d = 11$, $\mathbf{C}$ indicates that there is only one support for the collapse of element (4,4), thus its collapse should be prioritized.

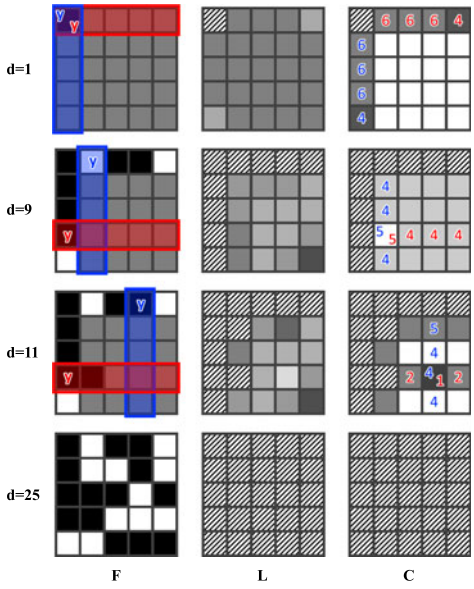


Fig. 8. An example of generating marker field. A 5×5 marker field with 1×5 tag shape is generated. Slashed elements are settled and thus not interested. The image of $\mathbf{C}$ is normalized. Blue and red numbers are the supports in the blue and red mask, that is, $\min(\mathbf{N}^0_{1,R_1,\mathbf{y}_{\text{blue}}}, \mathbf{N}^1_{1,R_1,\mathbf{y}_{\text{blue}}})$ and $\min(\mathbf{N}^0_{1,R_2,\mathbf{y}_{\text{red}}}, \mathbf{N}^1_{1,R_2,\mathbf{y}_{\text{red}}})$ respectively.

4 Fast Approach

A significant problem of bWFC is the high complexity. For an ID tag with N elements, the initial tag pool $\mathbf{P}$ has 2^N rows, introducing a great burden to the morphological operations in hit map calculation. In practice, bWFC becomes time-consuming when $N > 9$. We propose fast-bWFC to address this issue. The core idea is similar and the multi-way tree T is also adopted. However, the collapse is informed by risk metrics instead:

1) *Risk of DOF* R^{DOF}. Ignoring conflicts, an incomplete ID tag with N unknowns has 2^N possibilities, and the DOF (degree of freedom) is N. ID tags with less DOF are at more risk because they have fewer choices to avoid conflicts. We assume a cloud conflict for every incomplete ID tag, thus the tag risk is $1/2^N$. The risk of unknown $\mathbf{x}$, that is $R^{\text{DOF}}(\mathbf{x})$, equals the largest tag risk from all the ID tags containing $\mathbf{x}$.
2) *Risk of conflict* R^{conflict}. Focusing on an incomplete ID tag $\mathbf{K}$, we can search for possible conflict ID tag $\mathbf{K}'$ by aligning each ID tag pair and checking whether the aligned values are matched, or one of the two is u. If an unknown $\mathbf{x}$ of $\mathbf{K}$ matches v of $\mathbf{K}'$, assignment $\mathbf{F}(\mathbf{x}) \leftarrow 1 - v$ takes no risk, and $\mathbf{F}(\mathbf{x}) \leftarrow v$ takes a risk of $1/2^{N-1}$, that is, $R^{\text{conflict}}_{1-v} = 0$ and $R^{\text{conflict}}_{v} = 1/2^{N-1}$. One unknown $\mathbf{x}$ might be included in different ID tags, thus there might be multiple trials of matching. We assume that each trial is independent, thus the final $R^{\text{cross}}_v(\mathbf{x})$ equals $1 - \prod(1 - R^{\text{conflict}}_{v,n_{\text{trial}}}(\mathbf{x}))$, where n_{trial} means the $n^{\text{th}}_{\text{trial}}$ trial.

Since both R^{DOF} and R^{conflict} are relevant to N, elements with large R^{DOF} usually have large R^{conflict}. For speed, fast-bWFC first calculates R^{DOF} for all unknowns, and chooses one with the largest R^{DOF} to focus on. Then the collapse

Authorized licensed use limited to: Lakehead University. Downloaded on September 28,2026 at 21:34:37 UTC from IEEE Xplore. Restrictions apply.

suggestions are provided as

$$Pairs = \begin{cases} \{(\mathbf{f},0),(\mathbf{f},1)\} & \text{if} \quad R_0^{\text{conflict}}(\mathbf{f}) \geq R_1^{\text{conflict}}(\mathbf{f}) \\ \{(\mathbf{f},1),(\mathbf{f},0)\} & \text{if} \quad R_0^{\text{conflict}}(\mathbf{f}) < R_1^{\text{conflict}}(\mathbf{f}) \end{cases}, \tag{17}$$

where $\mathbf{f}$ is the focus. For further speed, when calculating R^{conflict}, we only search $\mathbf{K}'$ from the complete ID tags and the ID tags containing $\mathbf{x}$.

Algorithm 2. Fast-bWFC

Input: initial marker field $\mathbf{F}_{\text{initial}}$, tag shapes $\boldsymbol{\Omega}$
Output: marker field $\mathbf{F}$

$\mathbf{F} = \mathbf{F}_{\text{initial}}$
define an empty multi-way tree T, set $d = 0$
while $\mathbf{F}$ contains unknowns **do**
  if conflict happens **then**
    repeat
      $d \leftarrow d - 1$
      $\mathbf{F}(T_d^p.\text{index}) \leftarrow u$
      remove T_d^p from T_d
    until T_d is nonempty
  else
    calculate R^{DOF}, and select a peak $\mathbf{f}$
    calculate $R_0^{\text{conflict}}(\mathbf{f})$ and $R_1^{\text{conflict}}(\mathbf{f})$
    calculate $Pairs$, and assign to T_d
  end if
  $T_d^p \leftarrow$ the first node of T_d
  $\mathbf{F}(T_d^p.\text{index}) \leftarrow T_d^p.\text{value}$
  $d \leftarrow d + 1$
end while

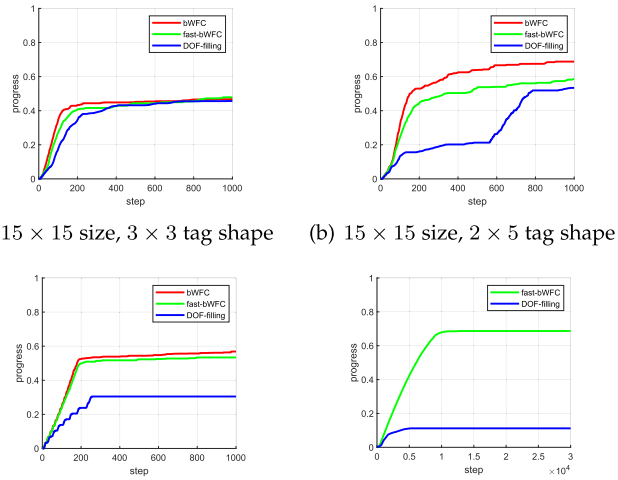
The algorithm is summarized in Algorithm 2. Since the risk metrics are lossy imitations of the superstate sensing metrics of bWFC, the assignment of fast-bWFC at each collapse is less precise. It is more likely to trigger rolling back. Even so, fast-bWFC is more practical because of the high speed.

5 Comparison

Orientable de Bruijn torus was first pursued by Burns and Mitchell [31], and the term marker field was first proposed by Szentandrasi et al. [35]. Recently, Narita et al. [36] and Gao et al. [27] composed fiducial markers based on marker field, and proposed their own generation algorithms.

The code of Gao et al. [27] is available and directly used in this section. Narita et al. [36] uses a brute-force strategy to search marker fields with four states. The method can only generate small-size marker fields, and the code is unavailable. Therefore, it is not discussed in comparison. Szentandrasi et al. [35] seems promising because their marker fields are much larger than previous methods. Unfortunately, the open-sourced code is not complete to be executable and we had to make assumptions to create functioning code to use in our comparisons. The implementation is also open-sourced as a function in HydraMarker toolkit.

Beside bWFC, fast-bWFC, Gao et al. [27], and Szentandrasi et al. [35], a baseline method named DOF-filling is included. DOF-filling blindly fills unknowns with the largest R^{DOF}. Compared to bWFC and fast-bWFC, it abandons most superstate sensing metrics, including $\mathbf{L}$, $\mathbf{C}$, and R^{conflict}, while the framework remains the same. Therefore, comparing it with bWFC and fast-bWFC helps to investigate the effectiveness of these metrics.



(a) 15×15 size, 3×3 tag shape (b) 15×15 size, 2×5 tag shape

(c) 15×15 size, 1×9 tag shape (d) 100×100 size, 4×4 tag shape

Fig. 9. Step efficiency comparison.

5.1 Step Efficiency

Step efficiency reflects the assignment accuracy of generating algorithm. The higher the accuracy, the smaller the possibility of triggering conflicts, thus the less the step it needs to complete a marker field. Gao et al. [27] and Szentandrasi et al. [35] do not have an element-wise filling process. Therefore, we only compare bWFC, fast-bWFC, and DOF-filling.

Four tests with different tag shapes are conducted. In each test, a marker field larger than possible is requested, and each method has a certain number of steps to pursue as much progress as possible. The progress is calculated by

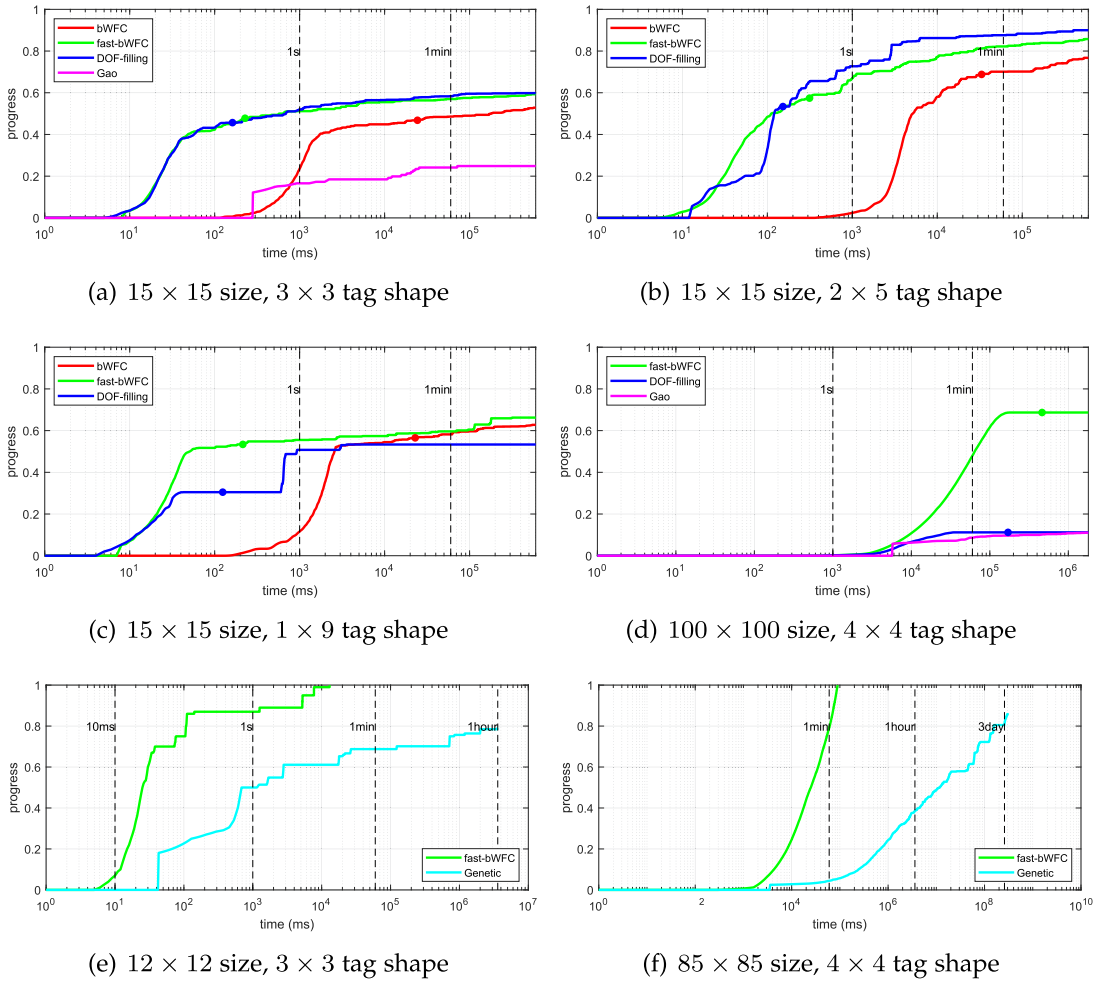
$$\text{progress} = \frac{\text{complete tags}}{\text{overall tags}}. \tag{18}$$

For example, an ordinary 3-order marker field with 10×10 size has $(10-2) \times (10-2) \times 4 = 256$ ID tags. If the left half unknowns are filled without conflict, the number of complete tags is $(10-2) \times (5-2) \times 4 = 96$, corresponding to a progress of 37.5%. Every method is tried 10 times in each test to produce an average progress/step curve. In each trial, we assign a random unknown of F_{initial} with 0 to make initial conditions different.

In the first test, we request an ordinary 3-order marker field with 15×15 size, and the step limit is 1000. The results are displayed in Fig. 9a. As can be seen, all three methods have similar limits. bWFC reaches the limit with the least steps. It assigns unknowns precisely. fast-bWFC takes slightly more steps. It is reasonable since fast-bWFC is a lossy imitation of bWFC. DOF-filling costs the most steps.

In the second test, a 2×5 tag shape is used instead. On the one hand, this test is easier because there are more usable tags. On the other hand, the test is slightly harder, because the ID tags are more complicatedly overlapped.

Authorized licensed use limited to: Lakehead University. Downloaded on September 28,2026 at 21:34:37 UTC from IEEE Xplore. Restrictions apply.



(a) 15×15 size, 3×3 tag shape

(b) 15×15 size, 2×5 tag shape

(c) 15×15 size, 1×9 tag shape

(d) 100×100 size, 4×4 tag shape

(e) 12×12 size, 3×3 tag shape

(f) 85×85 size, 4×4 tag shape

Fig. 10. Time efficiency comparison.

One assignment affects the unknowns up to 5 elements away. The results are displayed in Fig. 9b. Again, the rank of assignment accuracy is bWFC, fast-bWFC, and DOF-filling. Affected by the complicatedly overlapped tags, DOF-filling gets stuck in the early steps. As a comparison, bWFC and fast-bWFC are barely affected.

In the third test, a 1×9 tag shape is used. It offers less usable tags compared to a 2×5 tag shape but offers more compared to a 3×3 tag shape (180°-repeating tag shape has fewer self-conflicts compared to 90°-repeating tag shape). However, the ID tags are more complicatedly overlapped. The results are displayed in Fig. 9c. The rank of assignment accuracy repeats. DOF-filling is severely affected, resulting in a limit lower than the previous two.

The advantage of bWFC and fast-bWFC seems insignificant because DOF-filling can also complete many trials. Indeed, if the required marker field is small, we do not have to maximize the assignment accuracy. However, if the required marker field is large, it needs more than just luck to complete the lengthy filling process, and assignment accuracy becomes essential. In the fourth test, we request an ordinary 4-order marker field with 100×100 size, and the step limit is 30,000. A task of this giant size is much more challenging. bWFC is not available in this size because of its high complexity. The results of fast-bWFC and DOF-filling are displayed in Fig. 9d. As can be observed, DOF-filling barely moves, while fast-bWFC runs straight into a high limit.

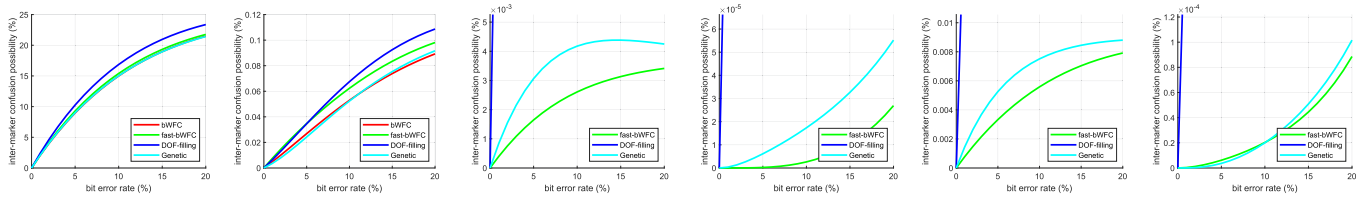
In brief, the superstate sensing metrics introduce significant assignment accuracy. bWFC is the best. fast-bWFC is less accurate, but the limit is similar. With the same framework but excluding most superstate sensing metrics, the assignment accuracy of DOF-filling is only satisfactory in the easiest test.

5.2 Time Efficiency

Time efficiency is more concerned in practice. However, the comparison is usually not satisfactorily rigorous, because most codes are not commercially optimized. We also face this problem, but fortunately, it does not affect the conclusion because these methods are at very different levels. The following tests were conducted on our laptop, Intel i9-11950H, 2.6 GHz CPU. All the codes are CPU-only, with no hardware acceleration such as parallelism or vectorization.

The four tests in Section 5.1 are repeated, but time limits are used instead. The first three tests are with time limits of 10 minutes and the fourth is with that of 30 minutes. The results are displayed from Figs. 10a to 10d, where the limits

Authorized licensed use limited to: Lakehead University. Downloaded on September 28,2026 at 21:34:37 UTC from IEEE Xplore. Restrictions apply.



(a) 3-order 10×10, read by 3×3 (b) 3-order 10×10, read by 4×4 (c) 4-order 14×14, read by 4×4 (d) 4-order 14×14, read by 5×5 (e) 4-order 20×20, read by 4×4 (f) 4-order 20×20, read by 5×5

Fig. 11. Error tolerance comparison.

of each method achieved in Section 5.1 are marked by points. In the first test, the best final progress is about 60%. It is enough to fill a 3-order marker field with 12×12 size, which is the theoretical upper bound [35]. In the fourth test, the best limit is about 70%, corresponding to a 4-order marker field with about 85×85 size.

From Figs. 10a to 10c, we can also see that fast-bWFC and DOF-filling are about 100 times faster than bWFC (check the x coordinates of the point marks, which indicate the progress at 1000 step). Such a great advantage enables them to surpass bWFC. DOF-filling even achieves the best in the second test. However, speed does not always work. In the third test, where ID tags are complicatedly overlapped, DOF-filling freezes twice, while bWFC increases stably. Despite the tortoise speed, bWFC goes further eventually. This phenomenon is more evident in the fourth test, where the gap between DOF-filling and fast-bWFC is significant.

Gao et al. [27] can only generate ordinary marker fields, thus it is only investigated in the first and the fourth tests. It is essentially a brute-force searching method, but it is faster than a naive implementation because it spares the time cost of conflict detection at each step by some complicated initializations. As can be seen from Figs. 10a and 10d, after initialization, Gao et al. [27] makes some progress quickly, but freezes at a low limit soon. It is inferior to the other three in 3-order marker field generation, and close to DOF-filling in 4-order marker field generation.

Szentandrasi et al. [35] searches ordinary marker fields based on genetic algorithm. It randomly initializes a large number of individual marker fields. Many conflicts exist in the individuals, but the average level is improved through the cycles of selection, crossover, and mutation. Since the individual maker fields are always fully filled, we can not measure the progress by Eq. (18). Instead, we use

$$\text{progress} = \frac{\text{non-conflict area of the best individual}}{\text{overall area}}. \quad (19)$$

For example, the overall area of a 10×10 marker field is 100. If the left half elements of the best individual are not included in any conflict ID tag, the non-conflict area is 50 and the progress is 50%. Because of the same reason, we can not request a size larger than possible. Instead, we request the sizes based on the final progress in the previous tests, that is, 12×12 for 3-order and 85×85 for 4-order. Szentandrasi et al. [35] is slow but steady, thus we give 1 h to the first test and about 3 days to the second. For the sake of time, each test is conducted only once.

The results are displayed in Figs. 10e and 10f. Szentandrasi et al. [35] fails both tests, but the curve in the second test has a high and steady slope. It seems that the 85×85 marker field can be finished at about the tenth day (the x-axis has a log scale). No surprise, this method is supposed to be executed on a supercomputer [35]. For the sake of time, we did not wait for the completion. The superiority of fast-bWFC is clear already, which uses about 15 seconds and 1.5 minutes respectively.

In brief, fast-bWFC is the best without a doubt. It has a good balance between speed and accuracy, resulting in the time efficiency at an entirely different level.

5.3 Memory Efficiency

We monitored the memory cost of each method but did not find a meaningful difference. All the methods cost no more than 40 MB for 3-order marker fields and no more than 100 MB for 4-order marker fields. Memories are mainly consumed by conflict detection, which needs to restore all the ID tags for comparison.

5.4 Error Tolerance

When reading makers built from marker fields, there is a possibility that some states are falsely recognized, leading to inter-window confusion [18]. The possibility of falsely reading a state is termed bit error rate, which depends on the device, environment, marker appearance, and reading algorithm. Given the rate, the smaller the expected number of conflict tags, the smaller the possibility of inter-window confusion, thus the better the error tolerance of the marker field.

In this section, we investigate the error tolerance quality of the marker fields generated by different methods. The expected number of conflict tags is calculated based on the same measure used in ARTag [18], that is

$$E = \sum_{n=1}^{N} H(n) p^n (1-p)^{N-n}, \quad (20)$$

where p is the bit error rate, N is the number of observed states, n is the number of falsely recognized states, H is the hamming distance histogram. The inter-window confusion possibility is the ratio of E and the overall tag number.

In the first test, 3-order marker fields with 10×10 size are generated 10 times by different methods. We assume the reading process only has a local region of 3×3 size, that is, $N = 9$. The inter-window confusion possibility curves are displayed in Fig. 11a. As can be seen, except

Authorized licensed use limited to: Lakehead University. Downloaded on September 28,2026 at 21:34:37 UTC from IEEE Xplore. Restrictions apply.

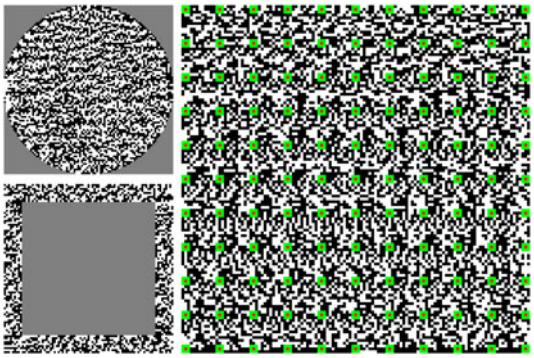


Fig. 12. Marker fields generated by fast-bWFC.

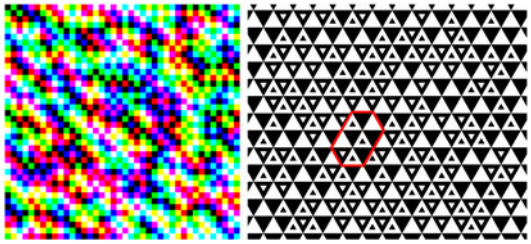


Fig. 13. Markers built from extended markers fields. Left, a chromatic marker whose every 3×3 squares are unique from all four orientations; Right, a triangle-based marker whose every hexagon pair is unique (a hexagon pair is outlined in red).

for DOF-filling which is obviously worse, the other three have similar qualities, where a 5% bit error rate introduces about 8% inter-window confusion possibility. It is a large number, but it is reasonable because a 10×10 marker field already includes more than half the usable tags. The larger the marker field, the less the space for error tolerance.

In practice, observed regions are usually larger than a single tag shape. For example, if the observed region is 4×4, we have $N = 16$, then the curves become significantly lower, as shown in Fig. 11b. With this slightly better visual condition, a 5% bit error rate introduces only about 0.03% inter-window confusion possibility.

In the second test, fast-bWFC and Szentandrasi et al. [35] are compared on 4-order marker fields. The results of Szentandrasi et al. [35] are downloaded from its project page. The curves of $N = 4 \times 4$ are displayed in Figs. 11c and 11e, and the curves of $N = 5 \times 5$ are displayed on the right. As shown, fast-bWFC performs obviously better in 14×14 size, and slightly better in 20×20 size. Both fast-bWFC and Szentandrasi et al. [35] are significantly better than DOF-filling, which rushes out of the coordinates. However, error tolerance is not concerned by these methods, then where does this superiority come from?

In each step, fast-bWFC counts existing tags and composes new tags whose states are contrary to most. It makes each tag very different from the others, maximizing their hamming distances. Although this is not a global optimization, it introduces good error tolerance when the marker field is relatively small. For Szentandrasi et al. [35], we attribute the superiority to the selection and mutation processes. If the ID tags of an individual are close, they have a larger possibility of triggering conflict after mutation, and the individual is more likely to lose the selection. The evolution process implicitly prefers individuals with better error tolerance.

5.5 Position-Sensing Ability

HydraMarker is the only advanced marker field generator (see the descriptions of *Flexible* and *Multifold* in Section 1). Some examples produced by fast-bWFC are shown in Fig. 12. From top-left to bottom-right:

1) A 90×90 circular marker field with 4×4 tag shape.
2) An 85×85 frame-shape marker field. The tag shapes are 4×4, 3×6, 2×9, and 1×20.
3) A 123×123 marker field with 5×5 tag shape. An array of locators are inserted. These insertions are part of the marker field, as red and green correspond to 0 and 1 respectively.

5.6 Shortcomings and Extensions

The rolling back mechanism of HydraMarker always starts from the end. If conflict happens due to wrong moves in early steps, it will take a lot of time to rectify. As a consequence, our methods can not maximize the marker field size. In the last test of Section 5.2, fast-bWFC achieves a 4-order 85×85 maker field. It only utilizes 41.2% ID tags. As a comparison, Szentandrasi et al. [35] achieves a size of 78×110, corresponding to a utilization rate of 49.2%. HydraMarker might be more efficient with a wiser process manager. We leave this problem to future works.

Actually, there is no inherent limitation of the state number and the arrangement of marker field elements. With some modifications on fast-bWFC, we achieved a chromatic marker and a triangle-based marker with also position-sensing ability, as displayed in Fig. 13. This is also a potential extension of HydraMarker. However, in this paper, we focus on presenting the efficiency and flexibility introduced by the idea of wave function collapse, thus we leave this topic to future works.

6 Conclusion

Marker field generation is a challenge of searching for feasible solutions in a great space under complex constraints. To solve this problem, we introduce the idea of wave function collapse, and implement it into a novel generating toolkit, named HydraMarker. By always assigning the unknowns which have the most certainty, we minimize the risk of every step. Experimental results show that, compared to state-of-the-arts, both the efficiency and the capability of HydraMarker are on an entirely higher level.

HydraMarker not only makes marker fields more powerful and flexible, but also makes marker field generation accessible to common marker designers. Users can highly customize their marker fields and generate them on laptops within a short period of time. We wish this work could introduce more possibilities for fiducial markers.

Authorized licensed use limited to: Lakehead University. Downloaded on September 28,2026 at 21:34:37 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] Z. Zhang, "A flexible new technique for camera calibration," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 22, no. 11, pp. 1330–1334, Nov. 2000.

[2] H. Asayama, D. Iwai, and K. Sato, "Fabricating diminishable visual markers for geometric registration in projection mapping," *IEEE Trans. Vis. Comput. Graph.*, vol. 24, no. 2, pp. 1091–1102, Feb. 2018.

[3] B. K. Kim, H. Tanaka, and Y. Sumi, "Robotic wheelchair using a high accuracy visual marker lentibar and its application to door crossing navigation," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2015, pp. 4478–4483.

[4] Z. Fan, G. Chen, J. Wang, and H. Liao, "Spatial position measurement system for surgical navigation using 3-D image marker-based tracking tools with compact volume," *IEEE Trans. Biomed. Eng.*, vol. 65, no. 2, pp. 378–389, Feb. 2018.

[5] A. Krupa et al., "Autonomous 3-d positioning of surgical instruments in robotized laparoscopic surgery using visual servoing," *IEEE Trans. Robot. Automat.*, vol. 19, no. 5, pp. 842–853, Oct. 2003.

[6] Y. Watanabe et al., "Extended dot cluster marker for high-speed 3D tracking in dynamic projection mapping," in *Proc. IEEE Int. Symp. Mixed Augmented Reality*, 2017, pp. 52–61.

[7] H. Chen, H. Park, K. Macit, and L. Kavan, "Capturing detailed deformations of moving human bodies," *ACM Trans. Graph.*, vol. 40, no. 4, pp. 1–18, 2021.

[8] H. Ha, M. Perdoch, H. Alismail, I. So Kweon, and Y. Sheikh, "Deltille grids for geometric camera calibration," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 5354–5362.

[9] D. Douxchamps and K. Chihara, "High-accuracy and robust localization of large control markers for geometric camera calibration," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 31, no. 2, pp. 376–383, Feb. 2009.

[10] M. Pérez, F. Agüera, and F. Carvajal, "Digital camera calibration using images taken from an unmanned aerial vehicle," *Int. Arch. Photogrammetry, Remote Sens. Spatial Inf. Sci.*, vol. 38, no. 1/C22, pp. 167–171, 2011.

[11] K. H. Strobl and G. Hirzinger, "More accurate pinhole camera calibration with imperfect planar targer," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2011, pp. 1068–1075.

[12] B. Atcheson, F. Heide, and W. Heidrich, "Caltag: High precision fiducial markers for camera calibration," in *Proc. Int. Workshop Vis., Model. Vis.*, 2010, pp. 41–48.

[13] Z. Xing, J. Yu, and Y. Ma, "A new calibration technique for multi-camera systems of limited overlapping field-of-views," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2017, pp. 5892–5899.

[14] P. Lébraly et al., "Fast calibration of embedded non-overlapping cameras," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2011, pp. 221–227.

[15] M. Fiala and C. Shu, "Self-identifying patterns for plane-based camera calibration," *Mach. Vis. Appl.*, vol. 19, no. 4, pp. 209–216, 2008.

[16] M. B. Yaldiz, A. Meuleman, H. Jang, H. Ha, and M. H. Kim, "Deepformabletag: End-to-end generation and recognition of deformable fiducial markers," *ACM Trans. Graph.*, vol. 40, no. 4, 2021, Art. no. 14.

[17] J. Rekimoto, "Matrix: A realtime object identification and registration method for augmented reality," in *Proc. Asia Pacific Comput. Hum. Interact.*, 1998, pp. 63–68.

[18] M. Fiala, "ARTag, a fiducial marker system using digital techniques," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2005, pp. 590–596.

[19] S. Garrido-Jurado, R. Munoz-Salinas, F. J. Madrid-Cuevas, and R. Medina-Carnicer, "Generation of fiducial marker dictionaries using mixed integer linear programming," *Pattern Recognit.*, vol. 51, pp. 481–491, 2016.

[20] L. Naimark and E. Foxlin, "Circular data matrix fiducial system and robust image processing for a wearable vision-inertial self-tracker," in *Proc. IEEE Int. Symp. Mixed Augmented Reality*, 2002, Art. no. 27.

[21] R. Bencina, M. Kaltenbrunner, and S. Jorda, "Improved topological fiducial tracking in the reactivision system," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops*, 2005, pp. 99–99.

[22] J. B. Peace, E. Psota, Y. Liu, and L. C. Perez, "E2etag: An end-to-end trainable method for generating and detecting fiducial markers," in *Proc. Brit. Mach. Vis. Conf.*, 2020, pp. 52–61.

[23] L. Calvet, P. Gurdjos, C. Griwodz, and S. Gasparini, "Detection and accurate localization of circular fiducials under highly challenging conditions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 562–570.

[24] F. Bergamasco, A. Albarelli, L. Cosmo, E. Rodola, and A. Torsello, "An accurate and robust artificial marker based on cyclic codes," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 38, no. 12, pp. 2359–2373, Dec. 2016.

[25] G. Yu, Y. Hu, and J. Dai, "TopoTag: A robust and scalable topological fiducial marker system," *IEEE Trans. Vis. Comput. Graph.*, vol. 27, no. 9, pp. 3769–3780, Sep. 2021.

[26] A. Herout, I. Szentandrási, M. Zacharias, M. Dubská, and R. Kajan, "Five shades of grey for fast and reliable camera pose estimation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2013, pp. 1384–1390.

[27] Z. Gao, M. Zhu, and J. Yu, "A novel camera calibration pattern robust to incomplete pattern projection," *IEEE Sensors J.*, vol. 21, no. 8, pp. 10051–10060, Apr. 2021.

[28] C. Matsunaga and K. Kanatani, "Optimal grid pattern for automated matching using cross ratio," in *Proc. IAPR Workshop Mach. Vis. Appl.*, 2000, pp. 561–564.

[29] B. Li, L. Heng, K. Koser, and M. Pollefeys, "A multiple-camera system calibration toolbox using a feature descriptor-based calibration pattern," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2013, pp. 1301–1307.

[30] M. Fiala, "Designing highly reliable fiducial markers," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 32, no. 7, pp. 1317–1324, Jul. 2010.

[31] J. Burns and C. J. Mitchell, *Coding schemes for two-dimensional position sensing*, Palo Alto, CA, USA: Hewlett-Packard Laboratories, Technical Publications Department, 1992.

[32] H. Bay, A. Ess, T. Tuytelaars, and L. Van Gool, "Speeded-up robust features (surf)," *Comput. Vis. Image Understanding*, vol. 110, no. 3, pp. 346–359, 2008.

[33] G. K. Aguirre, M. G. Mattar, and L. Magis-Weinberg, "de bruijn cycles for neural decoding," *NeuroImage*, vol. 56, no. 3, pp. 1293–1300, 2011.

[34] B. Jackson, B. Stevens, and G. Hurlbert, "Research problems on gray codes and universal cycles," *Discrete Math.*, vol. 309, no. 17, pp. 5341–5348, 2009.

[35] I. Szentandrasi, M. Zachariáš, J. Havel, A. Herout, M. Dubska, and R. Kajan, "Uniform marker fields: Camera localization by orientable de bruijn tori," in *Proc. IEEE Int. Symp. Mixed Augmented Reality*, 2012, pp. 319–320.

[36] G. Narita, Y. Watanabe, and M. Ishikawa, "Dynamic projection mapping onto deforming non-rigid surface using deformable dot cluster marker," *IEEE Trans. Vis. Comput. Graph.*, vol. 23, no. 3, pp. 1235–1248, Mar. 2017.



Mingzhu Zhu received the BE, ME, and DE degrees from the School of Mechanical Engineering and Automation, Fuzhou University, Fuzhou, China, in 2012, 2015, and 2019, respectively. From 2019 to 2021, he was a postdoctoral research fellow with BIC-ESAT, College of Engineering, Peking University, Beijing, China. In 2021, he joined Fuzhou University, as an associate scientist. His current research interests include computer vision, image processing, and surgical navigation.



Bingwei He received the ME and DE degrees from the School of Mechanical Engineering and Automation, Xi'an Jiaotong University, Shan Xi, China, in 1999 and 2003, respectively. From 2004 to 2010, he was a research fellow with the City University of Hong Kong. From 2013 to 2015, he was a visiting scholar with the University of Hamburg, Germany. In 2005, he was a full professor with Fuzhou University, Fuzhou, China. His current research interests include computer vision, image processing, and surgical navigation.

Authorized licensed use limited to: Lakehead University. Downloaded on September 28,2026 at 21:34:37 UTC from IEEE Xplore. Restrictions apply.



Junzhi Yu (Fellow, IEEE) received the BE degree in safety engineering and the ME degree in precision instruments and mechanology from the North University of China, Taiyuan, China, in 1998 and 2001, respectively, and the PhD degree in control theory and control engineering from the Institute of Automation, Chinese Academy of Sciences, Beijing, China, in 2003. From 2004 to 2006, he was a postdoctoral research fellow with the Center for Systems and Control, Peking University, Beijing. In 2006, he was an Associate Professor with the Institute of Automation, Chinese Academy of Sciences, where he became a full professor, in 2012. In 2018, he joined the College of Engineering, Peking University, as a Tenured full professor. His current research interests include intelligent robots, motion control, and intelligent mechatronic systems.



Fusong Yuan received the BA degree in dentistry from Weifang Medical University, Shandong, China, in 2008, and the ME and PhD degrees in prosthodontics from Peking University, Beijing, China, in 2011 and 2014, respectively. In 2014, he joined the Peking university school and hospital of Stomatology, as a dentist, teacher, and researcher. In 2021, he was promoted to associate chief physician. His current research interest is robotic and femtosecond laser technology in stomatology.



Jiantao Liu received the ME and DE degrees from the University of Pittsburgh, USA and Fuzhou University, China, in 2019 and 2014, respectively. His main research interests are computer vison, machine learning, and computational mechanics. He is currently the director of artificial intelligence with Xiamen Port Holding Group. Previously, he was a researcher with MTlab of Meitu Inc and an AI engineer at Yealink.

▷ **For more information on this or any other computing topic, please visit our Digital Library at** www.computer.org/csdl.

Authorized licensed use limited to: Lakehead University. Downloaded on September 28,2026 at 21:34:37 UTC from IEEE Xplore. Restrictions apply.